

NexusMUD Design Companion

Architecture Reference & API Specification

Version: 0.2.0

Date: 2026-05-31

Status: **Living Document**

NexusMUD Internal Engineering Reference — Not for External Distribution

Table of Contents

Chapter 1: GroundTruth API Specification

- 1.1 Overview
- 1.2 Core Data Models
- 1.3 API Endpoints
- 1.4 Authentication & Authorization
- 1.5 Error Handling
- 1.6 Rate Limiting & Quotas
- 1.7 Versioning Strategy

Chapter 2: Event Envelope Specification

- 2.1 Overview & Design Goals
- 2.2 Event Envelope Schema (Full)
- 2.3 Field Definitions & Constraints
- 2.4 Typed Async Python Interface
- 2.5 Idempotency Rules
- 2.6 Causal Metadata & Lineage Tracking
- 2.7 Event Categories & Type Registry
- 2.8 Example Handlers: world_engine

2.9 Example Handlers: `agent_core`

2.10 Example Handlers: `gateway_api`

2.11 Versioning & Schema Evolution

2.12 Error & Dead-Letter Handling

Appendix A: Glossary

Appendix B: Future Chapters (Placeholder)

Document Revision History

Chapter 1: GroundTruth API Specification

1.1 Overview

GroundTruth is the authoritative source-of-truth service for all persistent world state in NexusMUD. It provides read and write access to entity state — rooms, players, NPCs, items, and zones — as well as world metadata and canonical game-rule references. All other services (`world_engine`, `agent_core`, `gateway_api`) treat GroundTruth as the single record of truth for durable state, caching locally only for performance optimization.

GroundTruth is intentionally narrow in scope: it owns persistence and consistency, but delegates business logic to the services that consume it. No game rules are enforced at the GroundTruth layer; it validates structure and version integrity only.

Design Principle:

GroundTruth is a data plane service, not a control plane. It stores facts; it does not

make decisions.

Key Design Principles

- **Append-only audit log** underpins all mutations. Every write creates an immutable audit record before the entity record is updated.
- **Optimistic concurrency** via ETags and version tokens. Concurrent writers must supply the current version; conflicts return `VERSION_CONFLICT`.
- **GraphQL query layer** for flexible, field-selective reads; REST endpoints for all writes and mutations.
- **Event emission on every state change** feeds into the event bus described in Chapter 2, ensuring downstream services remain consistent without polling.
- **Zone-scoped partitioning** aligns with the shard model described in the planned Chapter 6.

1.2 Core Data Models

All world objects in NexusMUD inherit from the `Entity` base type. Specific object types extend this base with domain-specific fields. The following tables define each model in full.

Entity (Base Type)

All world objects inherit from `Entity`. Fields defined here are present on every object stored in GroundTruth.

Field	Type	Constraints	Description
<code>entity_id</code>	UUID	PK, immutable, UUIDv4	Globally unique identifier. Assigned at creation; never reused or reassigned.

Field	Type	Constraints	Description
entity_type	enum	ROOM PLAYER NPC ITEM ZONE	Discriminant used by all consumers to route to the correct sub-schema.
version	int	Monotonic, starts at 1, incremented on every write	Optimistic concurrency token. Writers must supply the current version on PUT.
created_at	string	ISO 8601 UTC, immutable	Timestamp of entity creation. Set by GroundTruth on POST; read-only thereafter.
updated_at	string	ISO 8601 UTC, server-set	Timestamp of last mutation. Updated atomically with every write.
tags	list[str]	Max 64 tags, each max 128 chars	Arbitrary string labels for filtering and categorization. Queryable via GraphQL.
metadata	JSON object	Max 16 KB serialized, keys must be strings	Unstructured key-value store for service-specific annotations not covered by typed fields.

Room

Inherits all `Entity` fields. Represents a discrete navigable location within the game world.

Field	Type	Constraints	Description
zone_id	UUID	FK → Zone, required	The zone this room belongs to. Used for shard routing and access control.
name	str	Max 128 characters, required	Short display name shown in player output (e.g., "The Rusty Flagon").
description	str	Max 4096 characters, required	Full prose description rendered

Field	Type	Constraints	Description
			to players on room entry or LOOK command.
exits	map[Direction → UUID]	Direction enum: N S E W NE NW SE SW UP DOWN IN OUT	Maps exit directions to the entity_id of the destination Room.
flags	set[enum]	DARK SAFE UNDERWATER INDOORS OUTDOORS	Behavioral flags affecting mechanics (e.g., DARK disables sight, SAFE prevents combat).
coordinates	object	x: int, y: int, z: int; all required if present	Optional 3D grid coordinates for map rendering and spatial queries.

Player

Inherits all `Entity` fields. Represents an active player character (PC) within the game world.

Field	Type	Constraints	Description
account_id	UUID	FK → Account service, required	Links the player entity to the authentication account. One account may have multiple characters.
display_name	str	Max 64 chars, unique within realm, required	The player's in-game name shown to other players.
location_id	UUID	FK → Room, required	The Room the player currently occupies. Updated on every successful move.
stats	object	Fields: hp (int≥0), max_hp (int>0), mp (int≥0), max_mp (int≥0), stamina (int 0-100)	Core vitals. hp reaching 0 triggers the death sequence.
inventory	list[UUID]	FK → Item, max 50 items	Ordered list of Item entity_ids the player is carrying.
equipped	map[slot → UUID]	Slot enum: HEAD	Maps equipment

Field	Type	Constraints	Description
		CHEST LEGS FEET HANDS MAIN_HAND OFF_HAND RING NECK	slots to the Item entity_id currently worn or wielded.
status_effects	list[StatusEffect]	StatusEffect: {effect_id, name, expires_at, magnitude}	Active buffs, debuffs, and conditions affecting the player's mechanics.

NPC

Inherits all `Entity` fields. Represents a non-player character managed by `agent_core`.

Field	Type	Constraints	Description
template_id	UUID	FK → NPC template library, required	Reference to the static template that defines this NPC's base attributes and behavior.
spawn_zone_id	UUID	FK → Zone, required	The zone responsible for spawning and respawning this NPC instance.
behavior_tree	str	Asset reference path, max 256 chars	Path to the behavior tree (BT) asset that governs this NPC's decision-making.
loot_table_id	UUID	FK → loot table registry, nullable	Loot table applied when this NPC is defeated.
faction	enum	NEUTRAL HOSTILE FRIENDLY GUARD MERCHANT QUEST_GIVER	Determines default disposition toward player characters and other NPCs.
aggro_radius	int	Distance in meters, min 0, max 100	Radius within which this NPC auto-aggresses hostile targets.
current_path	list[UUID]	FK → Room, max 32 waypoints	Ordered list of Room entity_ids representing the NPC's current patrol or navigation path.

Item

Inherits all `Entity` fields. Represents any object that can exist in the game world, a player's inventory, or an equipment slot.

Field	Type	Constraints	Description
<code>item_type</code>	enum	WEAPON ARMOR CONSUMABLE QUEST MISC	Determines which sub-schema applies within the <code>properties</code> field.
<code>weight</code>	float	Min 0.0, max 1000.0, in kg	Physical weight. Contributes to carry weight calculations against the player's stamina.
<code>value</code>	int	Min 0, in gold coins	Base trade value. Merchant pricing may apply multipliers.
<code>owner_id</code>	UUID	FK → Entity, nullable	The entity (Player or NPC) that currently possesses this item. Null if on the ground.
<code>equipped_by</code>	UUID	FK → Entity, nullable	The entity that currently has this item equipped. Null if in inventory or on the ground.
<code>properties</code>	JSON object	Schema varies by <code>item_type</code> ; max 8 KB serialized	Type-specific attributes. WEAPON: { <code>damage_min</code> , <code>damage_max</code> , <code>damage_type</code> , <code>speed</code> }. ARMOR: { <code>defense</code> , <code>slot</code> , <code>resistances</code> }. CONSUMABLE: { <code>effect</code> , <code>duration</code> , <code>uses_remaining</code> }.

1.3 API Endpoints

All endpoints are prefixed with the base URL

<https://groundtruth.nexusmud.internal>. All requests and responses use application/json content type. The ETag response header carries the current entity version on all GET responses; PUT requests must supply If-Match: <version>.

Method	Path	Description	Auth Scope	Rate Limit
GET	/v1/entities/{entity_id}	Retrieve a single entity by its UUID. Returns the full typed model including all inherited Entity fields. Supports ?fields= for partial projection.	groundtruth:read	1000 req/min
PUT	/v1/entities/{entity_id}	Full replacement update of a single entity. Caller must supply If-Match header with current version. Increments version atomically. Returns updated entity.	groundtruth:write	500 req/min
POST	/v1/entities	Create a new entity. entity_id is server-assigned. entity_type must be provided. Returns 201 with created entity and Location header.	groundtruth:write	200 req/min
DELETE	/v1/entities/{entity_id}	Soft-delete an entity. Sets a deleted_at timestamp; entity is retained in audit log. Hard-delete requires	groundtruth:write	100 req/min

Method	Path	Description	Auth Scope	Rate Limit
		groundtruth:admin scope and separate endpoint.		
GET	/v1/rooms/{room_id}/occupants	List all entities currently located in the specified room. Returns a paginated list of entity_ids with entity_type. Supports ?type=PLAYER NPC filter.	groundtruth:read	1000 req/min
POST	/v1/players/{player_id}/move	Atomically move a player from their current room to a specified destination room. Validates exits, SAFE flags, and encumbrance. Emits world.room_exited and world.room_entered events.	groundtruth:write	300 req/min
GET	/v1/zones/{zone_id}/state	Retrieve aggregate state snapshot for an entire zone: all rooms, NPC positions, active players, and zone-level flags. Optimized for shard migration and world_engine bootstrap.	groundtruth:read	60 req/min
POST	/v1/items/transfer	Atomically transfer one or more items between entities (player-to-player, player-to-room, NPC-to-player, etc.). Body: {item_ids,	groundtruth:write	200 req/min

Method	Path	Description	Auth Scope	Rate Limit
		from_entity_id, to_entity_id}. Validates ownership and capacity.		
GET	/v1/audit-log	Query the append-only audit log. Params: entity_id (required), since (ISO 8601), limit (max 500, default 50). Returns ordered list of audit records with actor, operation, before/after snapshots.	groundtruth:admin	120 req/min
POST	/graphql	GraphQL query endpoint for flexible, field-selective reads. Supports nested entity resolution, filtering, and pagination. Maximum query complexity: 50 units. Mutations are not supported via GraphQL — use REST write endpoints.	groundtruth:read	200 req/min; 50 complexity units/query

1.4 Authentication & Authorization

JWT Bearer Tokens

All client-to-service requests must supply a valid JWT Bearer token in the `Authorization` header. Tokens are issued by the NexusMUD identity service, carry a 15-minute expiry (`exp` claim), and must be refreshed using the identity service's

refresh endpoint before expiry. Token payloads include a `sub` (service or player account UUID), `iss` (identity service URL), and a `scopes` array.

Scopes

Scope	Grants	Typical Holders
<code>groundtruth:read</code>	GET and GraphQL query access to all non-admin endpoints	<code>world_engine</code> , <code>agent_core</code> , <code>gateway_api</code> , <code>analytics</code>
<code>groundtruth:write</code>	POST, PUT, DELETE on entity endpoints; all write mutations	<code>world_engine</code> , <code>scheduler</code>
<code>groundtruth:admin</code>	Audit log access, hard-delete, bulk operations, schema migrations	Platform ops, DM tooling

Service-to-Service mTLS

Internal service-to-service calls use mutual TLS (mTLS) client certificates in addition to JWT. Each service is provisioned a certificate by the internal CA. The certificate's CN maps to a service identity that is cross-referenced against the token's `sub` claim. mTLS certificates rotate on a 90-day schedule via the platform's certificate management service.

Per-Entity ACL Overrides

DM-controlled zones and admin-flagged entities may carry per-entity access control lists (ACLs) stored in the `metadata.__acl` field. When present, GroundTruth evaluates the ACL after scope validation. ACL entries support ALLOW and DENY rules keyed by service identity or account UUID. This mechanism is used to restrict player access to hidden areas, admin rooms, and in-development content.

1.5 Error Handling

All GroundTruth error responses follow a standard error envelope regardless of the HTTP status code. Consumers must check the `error.code` field for programmatic handling — do not parse `error.message`, as it is human-readable and subject to change.

Standard Error Envelope

JSON — GroundTruth Error Response

```
{  "error": {    "code": "ENTITY_NOT_FOUND",    "message": "Entity 'abc-123' does not exist.",    "trace_id": "4f2a1b8c-9d3e-4f6a-b2c1-7e8d9f0a1b2c",    "timestamp": "2026-05-31T13:08:00Z"  } }
```

Error Code Reference

Error Code	HTTP Status	Description	Recommended Consumer Action
ENTITY_NOT_FOUND	404	The requested entity_id does not exist or has been soft-deleted.	Do not retry. Verify entity_id correctness; check if entity was recently deleted via audit log.
VERSION_CONFLICT	409	The supplied If-Match version does not match the current entity version. Concurrent write detected.	Re-fetch the entity, apply business logic again, and retry the PUT with the updated version.
UNAUTHORIZED	401 / 403	401: Token missing, expired, or invalid. 403: Valid token but insufficient scope or ACL denial.	401: Refresh token and retry. 403: Do not retry; escalate scope request if legitimate.
RATE_LIMITED	429	The requesting service has exceeded its rate limit. Check x-RateLimit-Reset for retry time.	Back off until x-RateLimit-Reset. Implement exponential backoff with jitter for sustained bursts.
VALIDATION_ERROR	422	Request body failed schema or business rule validation. The message field contains field-level details.	Do not retry without modifying the request. Log the full error for debugging.
INTERNAL_ERROR	500	Unexpected server-side error. The trace_id can be used to locate server logs.	Retry with exponential backoff (max 3 times). Alert on-call if sustained beyond 60 seconds.

1.6 Rate Limiting & Quotas

Default Tier Limits

Parameter	Value	Scope
Standard request rate	1,000 req/min	Per service identity (JWT sub)
Burst allowance	2× standard rate for up to 10 seconds	Per service identity
GraphQL query complexity	50 complexity units per request	Per request
Audit log page size	Max 500 records per request	Per request
Zone state snapshot	60 req/min	Per service identity

Rate Limit Response Headers

All responses include the following headers regardless of whether the request was rate-limited:

- `X-RateLimit-Limit` — The maximum number of requests allowed per minute for this service identity.
- `X-RateLimit-Remaining` — The number of requests remaining in the current one-minute window.
- `X-RateLimit-Reset` — Unix timestamp (seconds) at which the current window resets.

GraphQL Complexity Calculation

Query complexity is calculated at parse time using the following rules: each root field costs 1 unit; each nested field costs 1 unit; list fields with pagination cost 2 units plus 1 per requested item in the limit argument. Queries exceeding 50 complexity units are rejected with a 422 `VALIDATION_ERROR` before execution. Persisted queries (registered at deploy time) are exempt from the per-request complexity limit.

1.7 Versioning Strategy

URL-Based Major Versioning

All GroundTruth REST endpoints include the major version number in the URL path (e.g., `/v1/entities/`, `/v2/entities/`). Multiple major versions may be served concurrently during the deprecation window. Clients must explicitly migrate to new major versions.

Change Classification

Change Type	Classification	Policy
New optional fields in responses	Non-breaking	Additive — deployed freely, no version bump required
New optional query parameters	Non-breaking	Additive — deployed freely
New endpoints	Non-breaking	Additive — deployed freely under current major version
Removing or renaming fields	Breaking	Requires new major version; 6-month deprecation window on prior version
Changing field types or semantics	Breaking	Requires new major version; migration guide required
Changing error codes or HTTP status	Breaking	Requires new major version

Deprecation Process

1. A new major version is published alongside the existing version.
2. The existing version receives a `Deprecation` response header indicating the sunset date.
3. All service teams are notified via the internal engineering changelog at `/docs/changelog.md`.
4. A 6-month migration window is provided. Services that have not migrated receive escalating alerts at the 3-month and 5-month marks.
5. On the sunset date, the deprecated version returns 410 Gone for all endpoints.

Chapter 2: Event Envelope Specification

2.1 Overview & Design Goals

The NexusMUD event bus is the nervous system of the platform. Every state change, player action, NPC behavior, and system signal is communicated as a structured event. The **Event Envelope** is the universal wrapper that all events must conform to, regardless of their payload type. This ensures consistent routing, observability, idempotency enforcement, and causal tracking across all services.

The event bus is architected for high-throughput, low-latency delivery with at-least-once semantics. All consumers must implement idempotency guards (see Section 2.5). The bus is the sole mechanism for cross-service communication; direct service-to-service REST calls are reserved for synchronous request-response flows where a result is needed inline.

Architectural invariant:

No service may read another service's internal database. All cross-service data flows through either the GroundTruth API (Chapter 1) or the event bus (this chapter).

Design Goals

Goal	Description
Uniform schema	Every event from any producer is structurally identical at the envelope level. Payload differences are contained within

Goal	Description
	the typed payload field.
Typed payloads	payload is a strongly-typed discriminated union keyed by event_type. Consumers deserialize to the concrete type matching the event they handle.
Idempotency by default	Each event carries a globally unique envelope_id. Consumers must deduplicate on this field (or the optional idempotency_key). The broker guarantees at-least-once delivery.
Causal lineage	Every event optionally references its causing event(s) via causation_id and causal_chain, enabling full trace trees for debugging, replay, and audit.
Schema evolution	Additive field changes are non-breaking. Payload versioning uses a semver string (event_version). Consumers gate on this for migration compatibility.
Dead-letter support	Unprocessable events are routed to a dead-letter queue (DLQ) with enriched diagnostics, enabling manual inspection and replay.
Ordered delivery	Events with the same shard_key are delivered in publication order within that shard. Cross-shard ordering is not guaranteed and must not be assumed.

2.2 Event Envelope Schema (Full)

The following JSON Schema (draft-2020-12 compatible) is the normative definition of the Event Envelope. All producers must emit envelopes that validate against this schema. All consumers must tolerate unknown optional fields (for forward compatibility).

JSON Schema — [schemas/event-envelope/v1.json](#)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://nexusmud.internal/schemas/event-envelope/v1.json",
  "title": "EventEnvelope",
  "type": "object",
  "required": [
    "envelope_id",
    "event_type",
    "event_version",
    "producer_id",
    "producer_service",
    "occurred_at",
    "envelope_schema_version",
    "payload"
  ],
  "additionalProperties": false,
  "properties": {
    "envelope_id": {
      "type": "string",
      "format": "uuid",
      "description": "Globally unique identifier for this event instance (UUIDv7 recommended)."
    }
  }
}
```



```

},      "event_type": {      "type": "string",      "pattern": "^[a-z_]+\.[a-z_]+$",      "description": "Dot-namespaced event type. Format: <category>.<action>. E.g., 'world.room_entered'."      },      "event_version": {      "type": "string",      "pattern": "^\\d+\\.\\.\\d+\\.\\.\\d+$",      "description": "Semver string for the payload schema. Consumers gate on this for migration."      },      "producer_id": {      "type": "string",      "format": "uuid",      "description": "Unique identifier of the producing service instance (pod/process UUID, rotated on restart)."      },      "producer_service": {      "type": "string",      "enum": [      "world_engine", "agent_core", "gateway_api", "groundtruth", "scheduler", "analytics"      ],      "description": "Logical service name of the event producer."      },      "occurred_at": {      "type": "string",      "format": "date-time",      "description": "ISO 8601 UTC timestamp of when the event logically occurred."      },      "published_at": {      "type": "string",      "format": "date-time",      "description": "ISO 8601 UTC timestamp of when the event was placed on the bus. Set by broker."      },      "envelope_schema_version": {      "type": "string",      "pattern": "^\\d+\\.\\.\\d+\\.\\.\\d+$",      "description": "Version of the EventEnvelope schema itself. Current: 1.0.0."      },      "correlation_id": {      "type": "string",      "format": "uuid",      "description": "Optional. Ties all events in a single user request/transaction together."      },      "causation_id": {      "type": "string",      "format": "uuid",      "description": "Optional. The envelope_id of the immediately preceding event that caused this one."      },      "causal_chain": {      "type": "array",      "items": { "type": "string", "format": "uuid" },      "maxItems": 32,      "description": "Optional. Ordered list of envelope_ids from root cause to immediate parent."      },      "idempotency_key": {      "type": "string",      "maxLength": 256,      "description": "Optional. Business-level key for deduplication. If absent, envelope_id is used."      },      "session_id": {      "type": "string",      "format": "uuid",      "description": "Optional. Player session ID if event is player-initiated."      },      "actor_id": {      "type": "string",      "format": "uuid",      "description": "Optional. entity_id of the player, NPC, or system actor that triggered the event."      },      "zone_id": {      "type": "string",      "format": "uuid",      "description": "Optional. Zone context for geographic routing and partitioning."      },      "shard_key": {      "type": "string",      "description": "Optional. Explicit partition key for ordered delivery. Defaults to zone_id if present, else actor_id."      },      "ttl_seconds": {      "type": "integer",      "minimum": 0,      "description": "Optional. Maximum seconds this event is valid for processing. Expired events go to DLQ."      },      "flags": {      "type": "object",      "properties": {      "is_replay": { "type": "boolean", "default": false },      "requires_ack": { "type": "boolean", "default": false },      "is_confidential": { "type": "boolean", "default": false }      },      "additionalProperties": false      },      "tags": {      "type": "object",      "additionalProperties": { "type": "string" },      "description": "Optional key-value string tags for filtering, routing, and observability."      },      "payload": {      "type": "object",      "description": "Event-type-specific payload. Schema determined by event_type + event_version."      }      }
}

```

2.3 Field Definitions & Constraints

Field	Type	Required	Constraints	Description
envelope_id	UUID string	YES	UUIDv7 strongly recommended for time-sortability; UUIDv4 acceptable	Globally unique identifier for this event instance. Primary deduplication key for consumers.
event_type	string	YES	Pattern: <code>^[a-z_]+\.[a-z_]+\$</code>	Dot-namespaced event type. Left segment is the category; right segment is the action.
event_version	semver string	YES	Pattern: <code>^\d+\.\d+\.\d+\$</code>	Semver of the payload schema. Used by consumers for version-gated deserialization.
producer_id	UUID string	YES	Rotated on service restart; unique per process instance	Unique identifier of the producing pod or process. Used for debugging and tracing.
producer_service	enum string	YES	One of: <code>world_engine</code> , <code>agent_core</code> , <code>gateway_api</code> , <code>groundtruth</code> , <code>scheduler</code> , <code>analytics</code>	Logical service name. Used for routing, ACL enforcement, and metrics labeling.
occurred_at	ISO 8601 UTC	YES	Must be in UTC with Z suffix; must not be in the future	When the event logically occurred in the game world. Set by the producer.
published_at	ISO 8601 UTC	NO	Set by the broker, not the producer	When the event was accepted and committed to the bus. Used to measure publish lag.
envelope_schema_version	semver string	YES	Current value: <code>1.0.0</code>	Version of the EventEnvelope

Field	Type	Required	Constraints	Description
				schema itself. Allows envelope-level migration independent of payloads.
correlation_id	UUID string	NO	Should be propagated from the originating request	Ties all events belonging to a single logical transaction or user request together.
causation_id	UUID string	NO	Must reference a valid envelope_id if present	The envelope_id of the immediately preceding event that caused this one.
causal_chain	array[UUID]	NO	Max 32 elements; ordered root → immediate parent	Full ancestry chain for trace reconstruction. Truncated at 32 with a truncation flag.
idempotency_key	string	NO	Max 256 characters	Business-level deduplication key. When present, takes precedence over envelope_id for dedup.
session_id	UUID string	NO	Must reference an active session in gateway_api	Player session ID. Present on all player-initiated events for gateway_api fan-out.
actor_id	UUID string	NO	References an entity_id in GroundTruth	The entity that triggered the event. Used for ACL checks, logging, and NPC behavior logic.
zone_id	UUID string	NO	References a Zone entity in GroundTruth	Geographic context. Used as the default shard key if

Field	Type	Required	Constraints	Description
				shard_key is absent.
shard_key	string	NO	Max 256 characters; derived automatically if absent	Explicit partition key for ordered delivery. See routing rules below.
ttl_seconds	integer	NO	Minimum 0; no maximum enforced	Maximum age before the event is considered stale. Expired events are routed to the DLQ.
flags	object	NO	All sub-fields are boolean with default false	Control flags: is_replay (suppress side effects), requires_ack (explicit acknowledgment), is_confidential (PII present — restrict logging).
tags	map[str→str]	NO	String keys and values only; max 32 pairs	Arbitrary metadata for filtering, routing rules, and observability dashboards.
payload	object	YES	Schema varies by event_type + event_version; must not be null	The event-type-specific data. Deserialize using the concrete type mapped from event_type.

Routing & Partitioning

The shard_key controls which partition of the event bus receives an event, ensuring ordered delivery within a partition. The derivation rules are evaluated in priority order:

6. If shard_key is explicitly set by the producer, it is used as-is.

7. Otherwise, if `zone_id` is present, `shard_key` is derived as `zone:{zone_id}`.
8. Otherwise, if `actor_id` is present, `shard_key` is derived as `actor:{actor_id}`.
9. Otherwise, the broker assigns the event to a random partition (no ordering guarantee).

Producers in `world_engine` and `gateway_api` should always set `zone_id` or `shard_key` explicitly to ensure predictable ordering for game-critical events (movement, combat, stat changes).

Timestamp Semantics

`occurred_at` captures when the event *logically occurred in the game world* — the moment the player moved, the NPC made a decision, or the item was picked up. This timestamp is set by the producer and is meaningful for game logic, replay, and causal ordering.

`published_at` captures when the event was *committed to the event bus*. This is set by the broker and reflects infrastructure time. The delta between `occurred_at` and `published_at` is the publish lag metric. Both fields are required for observability: using only `published_at` would cause incorrect ordering during replay or when producer clocks drift; using only `occurred_at` would make infrastructure latency invisible to monitoring.

2.4 Typed Async Python Interface

The following Python module provides the canonical typed interface for producing and consuming NexusMUD events. All services must use or implement this interface. Python 3.12+ features are used throughout.

[Python 3.12 — nexusmud/events/envelope.py](#)

```
""" nexusmud.events.envelope ===== Canonical typed Event
Envelope for the NexusMUD event bus. Requires Python 3.12+. All services
share this module. """ from __future__ import annotations import re import
uuid from abc import ABC, abstractmethod from dataclasses import dataclass,
field from datetime import UTC, datetime from typing import Any, Awaitable,
Callable, Optional from uuid import UUID #
----- #
Constants #
-----
```

```

EVENT_TYPE_PATTERN = re.compile(r"^[a-z_]+\.[a-z_]+$") SEMVER_PATTERN =
re.compile(r"^\d+\.\d+\.\d+$") CURRENT_ENVELOPE_SCHEMA_VERSION = "1.0.0"
VALID_PRODUCER_SERVICES = frozenset({ "world_engine", "agent_core",
"gateway_api", "groundtruth", "scheduler", "analytics", })
MAX_CAUSAL_CHAIN_DEPTH = 32 #
----- #
Flags sub-object #
-----
@dataclass class EventFlags:      """Control flags embedded in every
EventEnvelope."""      is_replay: bool = False      """True when this event is
being replayed; consumers should suppress side effects."""      requires_ack:
bool = False      """True when the producer expects explicit acknowledgment
beyond broker ACK."""      is_confidential: bool = False      """True when
payload contains PII; suppress from standard logging pipelines."""      #
----- #
Payload base and concrete types #
-----
@dataclass class BasePayload(ABC):      """      Abstract base for all event
payloads. Concrete types must implement      to_dict() for serialization and
validate() for business-rule checks      beyond dataclass type enforcement.
"""      @abstractmethod      def to_dict(self) -> dict[str, Any]:
"""Serialize payload to a JSON-compatible dict."""      ...
@abstractmethod      def validate(self) -> None:      """Raise ValueError
if the payload violates business constraints."""      ...
@dataclass
class WorldRoomEnteredPayload(BasePayload):      """      Payload for
world.room_entered events.      Produced by world_engine when any entity
enters a room.      """      entity_id: UUID      """The entity (Player, NPC)
that entered the room."""      room_id: UUID      """The room that was
entered."""      previous_room_id: Optional[UUID]      """The room the entity
came from; None for initial spawn."""      direction: Optional[str]      """The
exit direction taken (e.g., 'N', 'UP'); None for teleport or spawn."""
zone_id: UUID      """Zone containing the destination room."""
entity_type: str      """'PLAYER' or 'NPC' – allows consumers to route without
a GroundTruth lookup."""      weather_state: Optional[str] = None
"""Optional (v1.1.0+). Current weather in the destination room's zone."""
def to_dict(self) -> dict[str, Any]:      return
{      "entity_id": str(self.entity_id),      "room_id":
str(self.room_id),      "previous_room_id": str(self.previous_room_id)
if self.previous_room_id else None,      "direction": self.direction,
"zone_id": str(self.zone_id),      "entity_type": self.entity_type,
"weather_state": self.weather_state,      }
def validate(self) ->
None:      if self.entity_type not in {"PLAYER", "NPC"}:      raise
ValueError(f"entity_type must be PLAYER or NPC, got {self.entity_type!r}")
if self.direction is not None:      valid_dirs =
{"N", "S", "E", "W", "NE", "NW", "SE", "SW", "UP", "DOWN", "IN", "OUT"}      if
self.direction not in valid_dirs:      raise
ValueError(f"direction {self.direction!r} not in {valid_dirs}")
@dataclass
class AgentDecisionPayload(BasePayload):      """      Payload for
agent.decision_made events.      Produced by agent_core when an NPC behavior
tree resolves a decision.      """      npc_id: UUID      """The NPC that made
the decision."""      behavior_tree_node: str      """The BT node name that
fired (e.g., 'PatrolAction', 'AttackTarget')."""      decision_type: str
"""High-level decision category: 'MOVE', 'ATTACK', 'DIALOG', 'IDLE',
'FLEE'."""      decision_data: dict[str, Any]      """Decision-specific
parameters. Schema varies by decision_type."""      confidence: float
"""Behavior tree utility score 0.0–1.0 that produced this decision."""
def to_dict(self) -> dict[str, Any]:      return {      "npc_id":
str(self.npc_id),      "behavior_tree_node": self.behavior_tree_node,
"decision_type": self.decision_type,      "decision_data":
self.decision_data,      "confidence": self.confidence,      }

```

```

def validate(self) -> None:
    valid_types = {"MOVE", "ATTACK", "DIALOG", "IDLE", "FLEE"}
    if self.decision_type not in valid_types:
        raise ValueError(f"decision_type must be one of {valid_types}")
    if not 0.0 <= self.confidence <= 1.0:
        raise ValueError(f"confidence must be in [0.0, 1.0], got {self.confidence}")
    # ----- #
Core EventEnvelope dataclass #
-----
@dataclass class EventEnvelope:
    """ Universal event wrapper for the NexusMUD event bus. Every event published to or consumed from the bus must be an instance of this class (or deserialize into one). Required fields must be supplied at construction. Optional fields default to None or type-appropriate defaults. __post_init__ performs validation; raises ValueError on failure. """
    # --- Required fields ---
    envelope_id: UUID = None
    """Globally unique event identifier. Use UUIDv7 (time-sortable) in production."""
    event_type: str = None
    """Dot-namespaced type string, e.g. 'world.room_entered'."""
    event_version: str = None
    """Semver payload version, e.g. '1.0.0'."""
    producer_id: UUID = None
    """UUID of the producing service instance (process-scoped)."""
    producer_service: str = None
    """Logical service name. Must be in VALID_PRODUCER_SERVICES."""
    occurred_at: datetime = None
    """UTC datetime of when the event logically occurred."""
    payload: dict[str, Any] = None
    """Serialized payload dict. Build via BasePayload.to_dict()."""
    # --- Optional fields ---
    published_at: Optional[datetime] = None
    """Set by the broker on commit; producers leave this None."""
    envelope_schema_version: str = CURRENT_ENVELOPE_SCHEMA_VERSION
    correlation_id: Optional[UUID] = None
    causation_id: Optional[UUID] = None
    causal_chain: list[UUID] = field(default_factory=list)
    idempotency_key: Optional[str] = None
    session_id: Optional[UUID] = None
    actor_id: Optional[UUID] = None
    zone_id: Optional[UUID] = None
    shard_key: Optional[str] = None
    ttl_seconds: Optional[int] = None
    flags: EventFlags = field(default_factory=EventFlags)
    tags: dict[str, str] = field(default_factory=dict)

    def __post_init__(self) -> None:
        """Validate all fields after dataclass initialization."""
        # event_type format
        if not EVENT_TYPE_PATTERN.match(self.event_type):
            raise ValueError(f"event_type {self.event_type!r} must match pattern '{EVENT_TYPE_PATTERN.pattern}' (lowercase, underscores only)")
        # semver on event_version and envelope_schema_version
        for version_field, value in [
            ("event_version", self.event_version),
            ("envelope_schema_version", self.envelope_schema_version),
        ]:
            if not SEMVER_PATTERN.match(value):
                raise ValueError(f"{version_field} {value!r} must be a valid semver string")
        # producer_service
        if self.producer_service not in VALID_PRODUCER_SERVICES:
            raise ValueError(f"producer_service {self.producer_service!r} not in {VALID_PRODUCER_SERVICES}")
        # occurred_at must be UTC-aware
        if self.occurred_at.tzinfo is None:
            raise ValueError("occurred_at must be timezone-aware (UTC)")
        # ttl_seconds
        if self.ttl_seconds is not None and self.ttl_seconds < 0:
            raise ValueError(f"ttl_seconds must be >= 0, got {self.ttl_seconds}")
        # causal_chain depth
        if len(self.causal_chain) > MAX_CAUSAL_CHAIN_DEPTH:
            raise ValueError(f"causal_chain exceeds maximum depth of {MAX_CAUSAL_CHAIN_DEPTH}")
        # idempotency_key length
        if self.idempotency_key is not None and len(self.idempotency_key) > 256:
            raise ValueError("idempotency_key must be <= 256 characters")
        @property
        def effective_idempotency_key(self) -> str:
            """Returns idempotency_key if set, otherwise envelope_id as string."""
            return self.idempotency_key or str(self.envelope_id)
        @property
        def

```

```

effective_shard_key(self) -> Optional[str]: """Derives shard_key per
routing rules if not explicitly set.""" if self.shard_key:
return self.shard_key if self.zone_id: return f"zone:
{self.zone_id}" if self.actor_id: return f"actor:
{self.actor_id}" return None def to_dict(self) -> dict[str,
Any]: """Serialize the full envelope to a JSON-compatible dict."""
return {
    "envelope_id": str(self.envelope_id),
    "event_type": self.event_type, "event_version":
self.event_version, "producer_id": str(self.producer_id),
    "producer_service": self.producer_service, "occurred_at":
self.occurred_at.isoformat(), "published_at":
self.published_at.isoformat() if self.published_at else None,
    "envelope_schema_version": self.envelope_schema_version,
    "correlation_id": str(self.correlation_id) if self.correlation_id else None,
    "causation_id": str(self.causation_id) if self.causation_id else None,
    "causal_chain": [str(c) for c in self.causal_chain],
    "idempotency_key": self.idempotency_key, "session_id":
str(self.session_id) if self.session_id else None, "actor_id":
str(self.actor_id) if self.actor_id else None, "zone_id":
str(self.zone_id) if self.zone_id else None, "shard_key":
self.shard_key, "ttl_seconds": self.ttl_seconds,
    "flags": {
        "is_replay": self.flags.is_replay,
        "requires_ack": self.flags.requires_ack, "is_confidential":
self.flags.is_confidential, }, "tags": self.tags,
    "payload": self.payload, } #
----- #
EventHandler type alias #
-----
EventHandler = Callable[[EventEnvelope], Awaitable[None]] """ Type alias for
async event handler functions. Every consumer registers handlers conforming
to this signature. """ #
----- #
EventBusProtocol #
-----
from typing import AsyncIterator, Protocol # noqa: E402 class
EventBusProtocol(Protocol): """ Protocol defining the interface all
event bus adapter implementations must conform to. Concrete
implementations exist for Kafka, NATS JetStream, and an in-process test
bus. """ async def publish(self, envelope: EventEnvelope) -> str:
""" Publish an EventEnvelope to the bus. Validates the
envelope schema before publishing. Returns the broker-assigned
message offset or ID. Raises: ValueError: If envelope
validation fails. EventBusError: If the broker rejects the
message. """ ... async def subscribe( self,
event_types: list[str], handler: EventHandler, ) ->
AsyncIterator[EventEnvelope]: """ Subscribe to one or more
event types and invoke handler for each. Delivers events to handler
with at-least-once semantics. The caller is responsible for
idempotency (see IdempotencyGuard). Args: event_types:
List of dot-namespaced event_type strings to subscribe to.
handler: Async callable invoked for each received envelope. Yields:
EventEnvelope instances as they are received. """ ... #
----- #
EnvelopeBuilder – fluent builder with method chaining #
-----
class EnvelopeBuilder: """ Fluent builder for EventEnvelope
instances. Ensures all required fields are supplied and auto-generates
envelope_id and occurred_at if not provided. Usage: envelope = (
EnvelopeBuilder()
    .for_type("world.room_entered",
version="1.0.0")
    .from_producer("world_engine",

```



```

producer_id=my_pod_uuid)                .caused_by(parent_envelope)
.with_payload(payload_obj)                .with_ttl(300)                .build()
)    """    def __init__(self) -> None:    self._event_type:
Optional[str] = None    self._event_version: Optional[str] = None
self._producer_service: Optional[str] = None    self._producer_id:
Optional[UUID] = None    self._occurred_at: Optional[datetime] = None
self._payload: Optional[dict[str, Any]] = None    self._correlation_id:
Optional[UUID] = None    self._causation_id: Optional[UUID] = None
self._causal_chain: list[UUID] = []    self._idempotency_key:
Optional[str] = None    self._session_id: Optional[UUID] = None
self._actor_id: Optional[UUID] = None    self._zone_id: Optional[UUID] =
None    self._shard_key: Optional[str] = None    self._ttl_seconds:
Optional[int] = None    self._flags: EventFlags = EventFlags()
self._tags: dict[str, str] = {}    def for_type(self, event_type: str,
version: str = "1.0.0") -> "EnvelopeBuilder":    """Set the event_type
and payload schema version."""    self._event_type = event_type
self._event_version = version    return self    def from_producer(
self,    service: str,    producer_id: Optional[UUID] = None,    )
-> "EnvelopeBuilder":    """Set the producer service and optional
process-scoped producer_id."""    self._producer_service = service
# In production, producer_id should be set once at service startup.    #
Fallback to uuid1 here as a UUIDv7 approximation (time-based).
self._producer_id = producer_id or uuid.uuid1()    return self    def
caused_by(self, parent: EventEnvelope) -> "EnvelopeBuilder":    """
Propagate causal metadata from a received parent envelope.    Sets
causation_id, correlation_id, and extends causal_chain.    """
self._causation_id = parent.envelope_id    self._correlation_id =
parent.correlation_id or parent.envelope_id    # Build new causal chain:
parent's chain + parent's own id    new_chain =
list(parent.causal_chain) + [parent.envelope_id]    # Truncate to
MAX_CAUSAL_CHAIN_DEPTH    if len(new_chain) > MAX_CAUSAL_CHAIN_DEPTH:
new_chain = new_chain[-MAX_CAUSAL_CHAIN_DEPTH:]    self._causal_chain =
new_chain    # Propagate context fields from parent    if
parent.zone_id and not self._zone_id:    self._zone_id =
parent.zone_id    if parent.session_id and not self._session_id:
self._session_id = parent.session_id    return self    def
with_payload(self, payload: BasePayload | dict[str, Any]) ->
"EnvelopeBuilder":    """Set the event payload. Accepts a BasePayload
instance or a raw dict."""    if isinstance(payload, BasePayload):
payload.validate()    self._payload = payload.to_dict()
else:    self._payload = payload    return self    def
with_ttl(self, seconds: int) -> "EnvelopeBuilder":    """Set the TTL in
seconds. Events older than TTL go to dead-letter."""
self._ttl_seconds = seconds    return self    def for_actor(self,
actor_id: UUID) -> "EnvelopeBuilder":    """Set the actor_id (entity
responsible for triggering the event)."""    self._actor_id = actor_id
return self    def in_zone(self, zone_id: UUID) -> "EnvelopeBuilder":
"""Set the zone_id context for geographic routing."""    self._zone_id =
zone_id    return self    def for_session(self, session_id: UUID) ->
"EnvelopeBuilder":    """Set the player session_id for player-initiated
events."""    self._session_id = session_id    return self    def
with_idempotency_key(self, key: str) -> "EnvelopeBuilder":    """Set a
business-level idempotency key."""    self._idempotency_key = key
return self    def with_tags(self, **tags: str) -> "EnvelopeBuilder":
"""Merge additional observability tags."""    self._tags.update(tags)
return self    def mark_replay(self) -> "EnvelopeBuilder":    """Mark
this event as a replay; consumers should suppress side effects."""
self._flags.is_replay = True    return self    def build(self) ->
EventEnvelope:    """    Construct and validate the EventEnvelope.
Raises ValueError if required fields are missing.    NOTE: envelope_id

```

```

uses uuid.uuid1() as a UUIDv7 approximation (time-sortable). In
production, replace with a proper UUIDv7 library such as 'uuid6' or
'uuid_utils'. """ if not self._event_type: raise
ValueError("event_type is required – call .for_type() first") if not
self._producer_service: raise ValueError("producer_service is
required – call .from_producer() first") if self._payload is None:
raise ValueError("payload is required – call .with_payload() first")
return EventEnvelope(
    envelope_id=uuid.uuid1(), # UUIDv7
    approximation
    event_type=self._event_type,
    event_version=self._event_version or "1.0.0",
    producer_id=self._producer_id or uuid.uuid1(),
    producer_service=self._producer_service,
    occurred_at=self._occurred_at or datetime.now(UTC),
    payload=self._payload,
    correlation_id=self._correlation_id,
    causation_id=self._causation_id,
    causal_chain=self._causal_chain,
    idempotency_key=self._idempotency_key,
    session_id=self._session_id,
    actor_id=self._actor_id,
    zone_id=self._zone_id,
    shard_key=self._shard_key,
    ttl_seconds=self._ttl_seconds,
    flags=self._flags,
    tags=self._tags,
) #
----- #
IdempotencyStore Protocol (abstract) #
-----
class IdempotencyStoreProtocol(Protocol): """Abstract interface for the
idempotency backing store (e.g., Redis).""" async def set_nx(self, key:
str, value: str, ttl_seconds: int) -> bool: """ Set key to
value with TTL only if key does not already exist. Returns True if
the key was newly set; False if it already existed. (Redis: SET key
value NX EX ttl) """ ... async def get(self, key: str) -
> Optional[str]: """Retrieve the value for key, or None if absent."""
... #
----- #
IdempotencyGuard #
-----
class IdempotencyGuard: """ Async guard for consumer-side event
deduplication. Backed by an IdempotencyStoreProtocol implementation
(e.g., Redis). Usage: guard = IdempotencyGuard(redis_store,
default_ttl_seconds=86400) if await guard.check_and_set(envelope):
await process(envelope) else: logger.info("Duplicate
event skipped", envelope_id=envelope.envelope_id) """ def __init__(
self, store: IdempotencyStoreProtocol, default_ttl_seconds:
int = 86400, # 24 hours recommended key_prefix: str =
"idempotency:", ) -> None: self._store = store
self._default_ttl = default_ttl_seconds self._prefix = key_prefix
async def check_and_set( self, envelope: EventEnvelope,
ttl_seconds: Optional[int] = None, ) -> bool: """ Check
whether this envelope has been processed before and mark it as seen.
Returns: True – First time this key is seen; caller SHOULD
process the event. False – Duplicate detected; caller MUST skip
processing. The idempotency key is derived from
envelope.effective_idempotency_key. TTL defaults to 24 hours
(configurable per consumer). IMPORTANT: The state mutation and this
check_and_set call must be committed atomically (same DB transaction
or two-phase pattern). See Section 2.5 for full atomicity rules.
""" key = f"{self._prefix}{envelope.effective_idempotency_key}"
ttl = ttl_seconds or self._default_ttl is_new = await
self._store.set_nx( key=key,
value=str(envelope.envelope_id), ttl_seconds=ttl, )
return is_new async def is_duplicate(self, envelope: EventEnvelope) ->
bool: """Convenience inverse of check_and_set for read-only duplicate

```

```
checks."""          key = f"{self._prefix}
{envelope.effective_idempotency_key}"      return (await
self._store.get(key)) is not None
```

2.5 Idempotency Rules

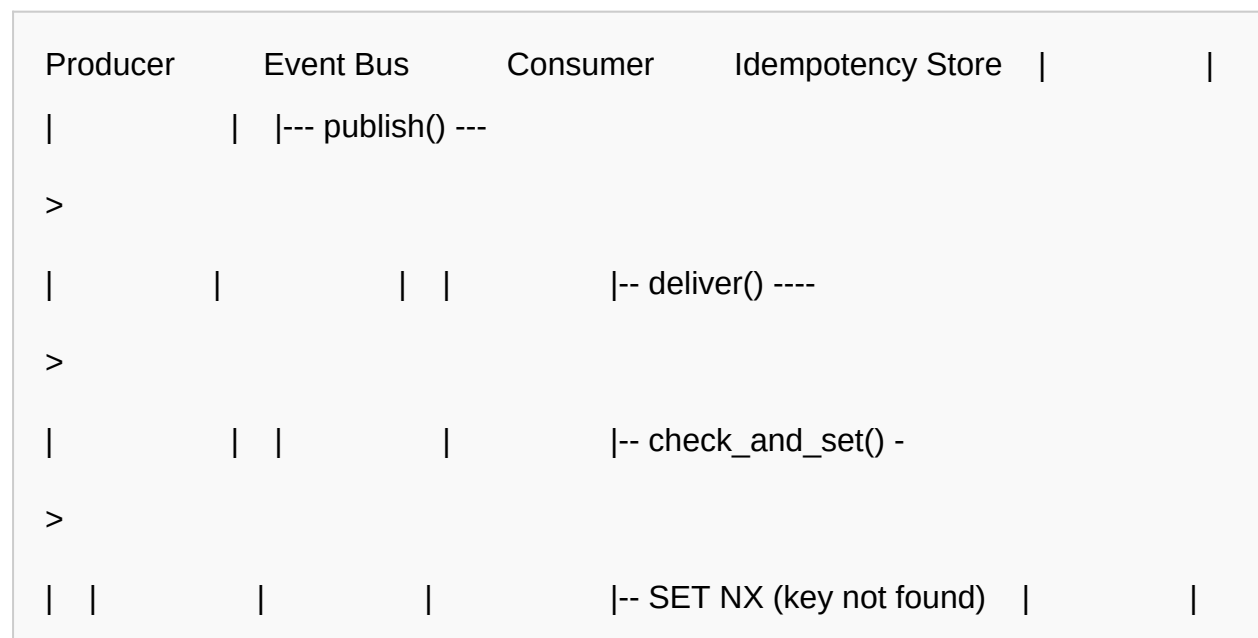
The event bus delivers messages with at-least-once semantics. Network partitions, consumer crashes, and broker retries can all result in the same event being delivered more than once. Every consumer *must* be idempotent. The following rules are normative.

10. **Rule 1 — Mandatory Deduplication Store:** Every consumer MUST track processed envelope identifiers for at least 24 hours. The deduplication key is `idempotency_key` if present; otherwise `envelope_id`. The store must be durable (Redis with AOF persistence or a SQL table) — an in-memory set is insufficient.
11. **Rule 2 — Detection Window:** The duplicate detection window is configurable per consumer with a minimum of 1 hour and a recommended default of 24 hours. Set the window to at least 2× the maximum expected broker retry interval for your consumer group.
12. **Rule 3 — Duplicate Handling:** On duplicate detection, the consumer MUST acknowledge the message to the broker without reprocessing the payload. It SHOULD emit an `idempotency.duplicate_skipped` meta-event containing the original `envelope_id`, the consumer service name, and the timestamp of the duplicate delivery. This enables observability of duplicate rates without re-triggering business logic.
13. **Rule 4 — Idempotency Key Construction:** Producers should use `envelope_id` (auto-generated) as the deduplication key for infrastructure and system events. For player-action events, producers SHOULD set an explicit `idempotency_key` using a business-meaningful string such as `player:{player_id}:cmd:{command_sequence_number}`. This prevents duplicate processing even if the producing service crashes and re-emits with a new `envelope_id`.

14. **Rule 5 — Delivery Guarantee:** The bus guarantees at-least-once delivery. Exactly-once delivery is not guaranteed at the broker level; consumers bear full responsibility for idempotency. Do not assume any event will be delivered exactly once, even under normal operating conditions.
15. **Rule 6 — Side-Effect Atomicity:** The state mutation and the idempotency mark MUST be committed atomically. Use one of two patterns: (a) Single-database transaction: write the idempotency key and the state change in the same DB transaction with a unique constraint on the key. (b) Two-phase pattern: first write the idempotency key to Redis with NX; if successful, apply the state mutation; if the mutation fails, delete the Redis key to allow retry.
16. **Rule 7 — Cross-Service Idempotency:** When processing an event triggers an outbound call to another service (e.g., a REST call to GroundTruth), include the original `idempotency_key` (or `envelope_id`) as the `Idempotency-Key` HTTP header on the downstream call. This ensures that even if the outbound call is retried, the downstream service performs the operation only once.

Delivery Sequence Diagrams

First Delivery (Processed Successfully):



```

|
<
---- True (new) ---|   |   |   |   |   |   |   |   |
|-- commit_state() |   |   |   |   |   |   |   |
<
--- ACK -----|   |   |   |   |   |   |   |

```

Duplicate Delivery (Skipped):

```

Producer      Event Bus      Consumer      Idempotency Store |   |
|             | |           |-- deliver() ----
>
| (re-delivery) | |           |-- check_and_set() -
>
| |             | |           |-- GET (key exists) |   |
|
<
---- False (dup) --|   |           |-- emit duplicate_skipped event |
|
<
--- ACK -----| (no reprocessing) | |           |   |

```

2.6 Causal Metadata & Lineage Tracking

Three-Field Causal Model

Field	Definition	When to Set
<code>correlation_id</code>	A single UUID that is shared by all events belonging to the same logical user request or business transaction.	Set by the first event in a chain (usually from the player's command or the triggering system event). All subsequent events in the same request must propagate this value unchanged.
<code>causation_id</code>	The <code>envelope_id</code> of the single, immediately preceding event that directly caused the current event.	Set whenever an event is produced in response to another event. A root event (no predecessor) leaves this field absent.
<code>causal_chain</code>	An ordered list of <code>envelope_ids</code> from the root cause through every ancestor to the immediate parent. Used to reconstruct the full trace without graph traversal.	Built by appending the parent's <code>envelope_id</code> to the parent's own <code>causal_chain</code> . The <code>EnvelopeBuilder.caused_by()</code> method handles this automatically.

Propagating Causal Metadata (Code Snippet)

When a consumer produces a downstream event in response to a received event, use the `caused_by()` builder method to correctly propagate all three causal fields:

Python 3.12 — Causal Propagation Pattern

```
async def handle_player_command(received: EventEnvelope) -> None: """
Example: world_engine handles player.command_received and produces
world.room_entered as a causal result. All causal metadata is propagated
via .caused_by(received). """ # ... validate and apply world state
mutation ... outgoing =
(
    EnvelopeBuilder()
    .for_type("world.room_entered",
version="1.0.0")
    .from_producer("world_engine",
producer_id=MY_POD_ID)
    .caused_by(received) # sets
causation_id, correlation_id, causal_chain
    .for_actor(player_id)
    .in_zone(zone_id)
    .with_payload(WorldRoomEnteredPayload(
entity_id=player_id,
room_id=destination_room_id,
previous_room_id=current_room_id,
direction=direction,
zone_id=zone_id,
```

```
entity_type="PLAYER",          ))          .with_ttl(30)          .build()          )
await bus.publish(outgoing)
```

Causal Chain Depth Limit

The `causal_chain` array has a hard maximum of 32 elements. When a chain would exceed this depth, the `EnvelopeBuilder.caused_by()` method truncates the oldest entries (preserving the most recent 32 ancestors). When truncation occurs, the tag `causal_chain_truncated: "true"` is automatically added to the envelope's `tags` field, alerting observability tooling to the incomplete chain. A truncated chain does not indicate an error condition — it is expected in long-running cascades such as patrol loops or sustained combat tick events.

OpenTelemetry Integration

EventEnvelope fields map to OpenTelemetry span attributes as follows, enabling correlation between event traces and distributed request traces:

Envelope Field	OTEL Span Attribute	Notes
envelope_id	nexusmud.event.envelope_id	Set as the span's trace.id for event-originated spans.
correlation_id	nexusmud.event.correlation_id	Inject into the W3C traceparent header for cross-service propagation.
causation_id	nexusmud.event.causation_id	Used as the parent span ID to build a trace tree in Jaeger/Tempo.
event_type	nexusmud.event.type	Also set as the span name for event-processing spans.
producer_service	service.name	Standard OTEL resource attribute; aligns with service mesh naming.
zone_id	nexusmud.zone.id	Enables zone-based trace filtering in dashboards.
actor_id	nexusmud.actor.id	Redact if flags.is_confidential is true.

Concrete Example: "attack goblin" Cascade

A player types `attack goblin`. This triggers a cascade of 5 events. The table below shows the full causal metadata for each event in the chain.

#	event_type	producer	envelope_id (abbreviated)	causation_id	causal_chain	correlation_id
1	player.command_received	gateway_api	aaa-001	— (root)	[]	aaa-001
2	world.combat_started	world_engine	bbb-002	aaa-001	[aaa-001]	aaa-001
3	agent.decision_made	agent_core	ccc-003	bbb-002	[aaa-001, bbb-002]	aaa-001
4	agent.goal_set	agent_core	ddd-004	ccc-003	[aaa-001, bbb-002, ccc-003]	aaa-001
5	player.stat_changed	world_engine	eee-005	bbb-002	[aaa-001, bbb-002]	aaa-001

Note that events 3 and 5 share the same `causation_id` (bbb-002) — both are direct children of the `world.combat_started` event, forming a branching causal tree, not a linear chain. All five share `correlation_id`: `aaa-001`, enabling reconstruction of the full transaction.

2.7 Event Categories & Type Registry

All event types are registered in this section. A new event type must be added to this registry before it may be produced. Unregistered event types are rejected at the schema validation layer.

Category	event_type	Payload Type	Producer	Consumers	Description
world	world.room_entered	WorldRoomEnteredPayload	world_engine	gateway_api, agent_core	Any entity (Player or NPC) has entered a room. Triggers NPC reactions and player session notifications.

Category	event_type	Payload Type	Producer	Consumers	Description
world.room_exited	WorldRoomExitedPayload	world_engine	gateway_api, agent_core	Any entity has left a room. Triggers NPC reaction checks and session updates for players in the vacated room.	
world.entity_spawned	EntitySpawnedPayload	world_engine	agent_core, analytics	A new entity has been created and placed in the world (NPC spawn, item drop via world mechanics).	
world.entity_despawned	EntityDespawnedPayload	world_engine	agent_core, analytics, gateway_api	An entity has been removed from the world (NPC death, item expiry). GroundTruth soft-delete is a prerequisite.	
world.combat_started	CombatStartedPayload	world_engine	agent_core, gateway_api	A combat encounter has begun. Payload includes all participants and the initiating actor.	
world.combat_ended	CombatEndedPayload	world_engine	agent_core, gateway_api, analytics	Combat has concluded. Payload includes outcome, survivors, and loot table reference for resolution.	
world.item_dropped	ItemDroppedPayload	world_engine	gateway_api, analytics	An item has been dropped into a room. Payload includes item_id, room_id, and the dropping entity.	
world.item_picked_up	ItemPickedUpPayload	world_engine	gateway_api, analytics	An entity has picked up an item from a room. Updates inventory in GroundTruth as a side effect.	
agent	agent.decision_made	AgentDecisionPayload	agent_core	world_engine	An NPC behavior tree has resolved a decision. world_engine applies the decision as a world state mutation.
	agent.dialog_started	AgentDialogPayload	agent_core	gateway_api	An NPC has begun a dialog session with a player. gateway_api enters dialog mode for the player session.
	agent.dialog_ended	AgentDialogPayload	agent_core	gateway_api	An NPC dialog session has concluded. gateway_api returns the player session to normal input mode.
agent.behavior	AgentBehavior	agent_core	analytics	An NPC has transitioned its	

Category	event_type	Payload Type	Producer	Consumers	Description
or_changed	orPayload			active behavior tree (e.g., idle → patrol → combat). Logged for AI tuning.	
agent.goal_set	AgentGoalPayload	agent_core	analytics	agent_core has assigned a goal to an NPC (e.g., "reach room X", "attack player Y").	
agent.goal_completed	AgentGoalPayload	agent_core	analytics, world_engine	An NPC has successfully completed an assigned goal. May trigger follow-on behavior tree evaluation.	
player	player.connected	PlayerConnectedPayload	gateway_api	world_engine, agent_core, analytics	A player has established a WebSocket session and authenticated. world_engine triggers spawn logic.
player.disconnected	PlayerDisconnectedPayload	gateway_api	world_engine, agent_core, analytics	A player session has terminated (graceful logout or connection loss). Triggers logout state cleanup.	
player.command_received	PlayerCommandPayload	gateway_api	world_engine, agent_core	A raw player command string has been parsed and validated. Downstream services apply game logic.	
player.stat_changed	PlayerStatPayload	world_engine	gateway_api, analytics	A player vital stat has changed (HP, MP, stamina). gateway_api pushes the delta to the player's client.	
player.level_up	PlayerLevelUpPayload	world_engine	gateway_api, analytics	A player has reached a new experience level. Triggers reward delivery, zone broadcast, and client notification.	
player.achievement_unlocked	PlayerAchievementPayload	world_engine	gateway_api, analytics	A player has satisfied the criteria for an achievement. Triggers badge award and optional zone announcement.	
system	system.shard_rebalanced	ShardRebalancePayload	world_engine	world_engine (peers), agent_core	Zone shard ownership has been reassigned. All services must update routing tables and

Category	event_type	Payload Type	Producer	Consumers	Description
					re-subscribe to zone topics.
system.service_heartbeat	HeartbeatPayload	any service	analytics, scheduler		Periodic liveness signal from each service instance. Used for health dashboards and dead-instance detection.
system.config_updated	ConfigPayload	groundtruth	all services		A runtime configuration value has been updated in GroundTruth. Services must reload affected config keys.
idempotency	idempotency.duplicate_skipped	DuplicateSkippedPayload	any consumer	analytics	A duplicate event was detected and skipped. Payload carries the original envelope_id and consumer identity.

2.8 Example Handlers: world_engine

Python 3.12 — [world_engine/handlers/player_command.py](#)

```

""" world_engine – player.command_received handler
===== Parses incoming player
commands, validates world state, applies mutations, and emits appropriate
world.* events causally. """
from __future__ import annotations
import logging
from dataclasses import dataclass
from uuid import UUID
from nexusmud.events.envelope import (
    EnvelopeBuilder, EventEnvelope,
    WorldRoomEnteredPayload, )
from nexusmud.groundtruth.client import
GroundTruthClient
from nexusmud.world_engine.command_parser import
CommandParser, ParsedCommand
from nexusmud.world_engine.state import
WorldStateCache
logger = logging.getLogger(__name__) # Process-scoped
producer UUID (set at service startup, rotated on restart)
MY_POD_ID: UUID = ... # injected from environment / service bootstrap
gt_client: GroundTruthClient = ... # injected dependency
bus: "EventBusProtocol" = ... # injected dependency
state_cache: WorldStateCache = ... # local in-memory
cache for hot path
async def handle_player_command_received(envelope:
EventEnvelope) -> None:
    """ Handler for player.command_received
    events. Responsibilities:
    1. Parse the raw command string into a
    structured ParsedCommand.
    2. Validate the action against current world
    state (GroundTruth + cache).
    3. Apply the state mutation atomically via
    GroundTruth API.
    4. Emit the appropriate world.* event with causal
    metadata propagated.
    """
    payload = envelope.payload
    player_id =

```

```

UUID(payload["player_id"])    raw_command: str = payload["raw_command"]
session_id = UUID(payload["session_id"])    logger.info("Received
player command",            player_id=player_id,            command=raw_command,
envelope_id=envelope_id,    )    # 1. Parse command    parsed:
ParsedCommand = CommandParser.parse(raw_command)    if parsed is None:
# Unknown command – emit no world event; gateway_api will send error
# to the player via a separate channel outside the event bus.
logger.warning("Unparseable command", command=raw_command,
player_id=player_id)    return    # 2. Validate against world state
if parsed.action == "MOVE":    await _handle_move(envelope, player_id,
session_id, parsed)    elif parsed.action == "TAKE":    await
_handle_take(envelope, player_id, session_id, parsed)    else:
logger.debug("Command action not handled by world_engine",
action=parsed.action)    async def _handle_move(    received: EventEnvelope,
player_id: UUID,    session_id: UUID,    parsed: "ParsedCommand", ) ->
None:    """Process a MOVE action: validate exit, commit move, emit room
events."""    direction: str = parsed.args["direction"].upper()    # Fetch
current player state (try cache first, fallback to GroundTruth)    player =
await state_cache.get_player(player_id) or \    await
gt_client.get_entity(player_id)    current_room_id =
UUID(player["location_id"])    # Fetch current room to validate exit exists
current_room = await state_cache.get_room(current_room_id) or \
await gt_client.get_entity(current_room_id)    if direction not in
current_room.get("exits", {}):    logger.info("Invalid move
direction",            player_id=player_id,            direction=direction,
room_id=current_room_id,    )    # No world event emitted;
gateway_api reads absence of world event    # and sends "You can't go
that way." via the session feedback channel.    return
destination_room_id = UUID(current_room["exits"][direction])
destination_room = await gt_client.get_entity(destination_room_id)
zone_id = UUID(destination_room["zone_id"])    # Validate SAFE/UNDERWATER
flags do not block movement (abbreviated)    room_flags =
set(destination_room.get("flags", []))    if "RESTRICTED" in room_flags:
return # silently blocked; gateway_api handles feedback separately    # 3.
Commit move atomically via GroundTruth    await gt_client.move_player(
player_id=player_id,    destination_room_id=destination_room_id,
idempotency_key=envelope_ikey(received), # cross-service idempotency    )
# Update local cache    state_cache.update_player_location(player_id,
destination_room_id)    # 4. Emit world.room_exited for the origin room
exited_evt =
(    EnvelopeBuilder()    .for_type("world.room_exited",
version="1.0.0")    .from_producer("world_engine",
producer_id=MY_POD_ID)    .caused_by(received)    .for_actor(player
_id)    .in_zone(UUID(current_room["zone_id"]))    .for_session(ses
sion_id)    .with_payload({    "entity_id": str(player_id),
"entity_type": "PLAYER",    "room_id": str(current_room_id),
"direction": direction,    "zone_id": str(current_room["zone_id"]),
})    .with_ttl(30)    .build()    )    await
bus.publish(exited_evt)    # 5. Emit world.room_entered for the destination
room    entered_payload =
WorldRoomEnteredPayload(    entity_id=player_id,
room_id=destination_room_id,    previous_room_id=current_room_id,
direction=direction,    zone_id=zone_id,    entity_type="PLAYER",
)    entered_evt =
(    EnvelopeBuilder()    .for_type("world.room_entered",
version="1.0.0")    .from_producer("world_engine",
producer_id=MY_POD_ID)    .caused_by(received)    .for_actor(player
_id)    .in_zone(zone_id)    .for_session(session_id)    .with
_payload(entered_payload)    .with_ttl(30)    .build()    )
await bus.publish(entered_evt)    logger.info("Player moved",

```

```

player_id=player_id, direction=direction,
destination=destination_room_id) def envelope_ikey(env: EventEnvelope) ->
str: """Derive a cross-service idempotency key from an envelope."""
return env.idempotency_key or str(env.envelope_id)

```

Python 3.12 — world_engine/handlers/agent_decision.py

```

""" world_engine - agent.decision_made handler
===== Applies NPC decisions produced by
agent_core to world state, then emits the corresponding world.* events with
causal lineage. """ from __future__ import annotations import logging from
uuid import UUID from nexusmud.events.envelope import EnvelopeBuilder,
EventEnvelope, WorldRoomEnteredPayload from nexusmud.groundtruth.client
import GroundTruthClient from nexusmud.world_engine.state import
WorldStateCache logger = logging.getLogger(__name__) MY_POD_ID: UUID = ...
gt_client: GroundTruthClient = ... bus: "EventBusProtocol" = ... state_cache:
WorldStateCache = ... async def handle_agent_decision_made(envelope:
EventEnvelope) -> None: """ Handler for agent.decision_made events.
agent_core has resolved an NPC behavior tree decision. This handler
applies the decision as a concrete world state mutation and emits the
appropriate world.* event, maintaining the causal chain. """ payload
= envelope.payload npc_id = UUID(payload["npc_id"]) decision_type:
str = payload["decision_type"] decision_data: dict =
payload["decision_data"] logger.info("Applying NPC decision",
npc_id=npc_id, decision_type=decision_type,
causation_id=envelope.causation_id, ) if decision_type == "MOVE":
await _apply_npc_move(envelope, npc_id, decision_data) elif decision_type
== "ATTACK": await _apply_npc_attack(envelope, npc_id, decision_data)
elif decision_type == "IDLE": # No world state change; optionally
emit an animation hint event pass else:
logger.warning("Unhandled NPC decision type", decision_type=decision_type)
async def _apply_npc_move( received: EventEnvelope, npc_id: UUID,
decision_data: dict, ) -> None: """ Apply an NPC MOVE decision:
commit new location to GroundTruth, emit world.room_exited and
world.room_entered with causal lineage tracing back through
agent.decision_made. """ destination_room_id =
UUID(decision_data["destination_room_id"]) direction =
decision_data.get("direction") # may be None for pathfinding # Fetch
NPC current state npc = await gt_client.get_entity(npc_id)
current_room_id = UUID(npc["location_id"]) destination_room = await
gt_client.get_entity(destination_room_id) zone_id =
UUID(destination_room["zone_id"]) # Commit NPC position update to
GroundTruth await gt_client.move_entity( entity_id=npc_id,
destination_room_id=destination_room_id,
idempotency_key=str(received.envelope_id), )
state_cache.update_entity_location(npc_id, destination_room_id) # Emit
world.room_exited for origin room exited_evt =
( EnvelopeBuilder().for_type("world.room_exited",
version="1.0.0").from_producer("world_engine",
producer_id=MY_POD_ID).caused_by(received) # causal chain:
player_cmd -> combat -> decision -> room_exit .for_actor(npc_id)
.in_zone(UUID(npc.get("zone_id", str(zone_id)))) .with_payload({
"entity_id": str(npc_id), "entity_type": "NPC",
"room_id": str(current_room_id), "direction": direction,
"zone_id": str(npc.get("zone_id",
str(zone_id))), }) .with_ttl(30) .build() )
await bus.publish(exited_evt) # Emit world.room_entered for destination
room entered_payload = WorldRoomEnteredPayload( entity_id=npc_id,
room_id=destination_room_id, previous_room_id=current_room_id,
direction=direction, zone_id=zone_id,

```

```

entity_type="NPC", ) entered_evt = ( EnvelopeBuilder()
.for_type("world.room_entered",
version="1.0.0") .from_producer("world_engine",
producer_id=MY_POD_ID) .caused_by(received) .for_actor(npc_id
) .in_zone(zone_id) .with_payload(entered_payload) .w
ith_ttl(30) .build() ) await bus.publish(entered_evt)
logger.info("NPC moved", npc_id=np_id, destination=destination_room_id)
async def _apply_npc_attack( received: EventEnvelope, np_id: UUID,
decision_data: dict, ) -> None: """Apply an NPC ATTACK decision –
placeholder for combat tick integration.""" # Combat resolution is
handled by the combat subsystem in world_engine. # This handler delegates
to CombatManager which emits player.stat_changed # events as a result.
Full implementation in Chapter 3 (World Engine Tick). target_id =
UUID(decision_data["target_id"]) logger.info("NPC attack decision
applied", np_id=np_id, target_id=target_id)

```

[Python 3.12 — world_engine/handlers/shard_rebalanced.py](#)

""" world_engine — system.shard_rebalanced handler

```

===== Handles zone shard
ownership reassignments. Updates local routing tables and re-subscribes to the new
zone's event topics. """ from __future__ import annotations import logging from uuid
import UUID from nexusmud.events.envelope import EventEnvelope from
nexusmud.world_engine.router import ZoneRouter from
nexusmud.world_engine.subscription_manager import SubscriptionManager logger =
logging.getLogger(__name__) zone_router: ZoneRouter = ... sub_manager:
SubscriptionManager = ... async def handle_shard_rebalanced(envelope:
EventEnvelope) -> None: """ Handler for system.shard_rebalanced events.
When the shard manager reassigns zone ownership: 1. Update the local zone routing
table to reflect new ownership. 2. Subscribe to event topics for newly assigned zones.
3. Unsubscribe from topics for zones transferred away. 4. Allow in-flight events for
transferred zones to drain gracefully. This handler is idempotent: applying the same
rebalance event twice results in the same routing table state. """ payload =
envelope.payload new_assignments: dict[str, str] = payload["zone_assignments"] #
{zone_id: owner_service_instance_id} our_instance_id: str =
payload.get("our_instance_id") or str(...) gained_zones: list[UUID] = [] lost_zones:
list[UUID] = [] for zone_id_str, owner_id in new_assignments.items(): zone_id =
UUID(zone_id_str) currently_owned = zone_router.is_owner(zone_id)

```

```

now_owned = (owner_id == our_instance_id)      if now_owned and not
currently_owned:      gained_zones.append(zone_id)      elif currently_owned and
not now_owned:      lost_zones.append(zone_id)      # 1. Update routing table
atomically      zone_router.apply_rebalance(new_assignments, our_instance_id)
logger.info("Zone routing table updated", gained=gained_zones, lost=lost_zones)      #
2. Subscribe to newly assigned zone topics      for zone_id in gained_zones:      await
sub_manager.subscribe_zone(zone_id)      logger.info("Subscribed to zone topics",
zone_id=zone_id)      # 3. Gracefully drain and unsubscribe from transferred zones.      #
Allow 5 seconds for in-flight events to complete before unsubscribing.      for zone_id in
lost_zones:      await sub_manager.drain_and_unsubscribe_zone(zone_id,
drain_seconds=5)      logger.info("Unsubscribed from zone topics", zone_id=zone_id)

```

2.9 Example Handlers: agent_core

[Python 3.12 — agent_core/handlers/room_entered.py](#)

```

""" agent_core – world.room_entered handler
===== When any entity enters a room,
agent_core evaluates whether NPCs present in that room should react, and
emits agent.decision_made if a behavior tree fires. """
from __future__
import annotations
import logging
from uuid import UUID
from
nexusmud.events.envelope import AgentDecisionPayload, EnvelopeBuilder,
EventEnvelope
from nexusmud.groundtruth.client import GroundTruthClient
from
nexusmud.agent_core.behavior import BehaviorTreeEngine
logger =
logging.getLogger(__name__)
MY_POD_ID: UUID = ...
gt_client:
GroundTruthClient = ...
bus: "EventBusProtocol" = ...
bt_engine:
BehaviorTreeEngine = ...
async def handle_world_room_entered(envelope:
EventEnvelope) -> None:
    """ Handler for world.room_entered events in
agent_core. When a PLAYER enters a room, check all NPCs in that room for
reaction behavior trees. For each NPC that fires, emit
agent.decision_made with the resolved decision and full causal lineage.
NPC-enters-room events are ignored here (NPC movements are already the
result of a prior agent.decision_made – no need to re-evaluate). """
    payload = envelope.payload
    entity_type: str = payload.get("entity_type",
    "")
    if entity_type != "PLAYER":
        return # Only player arrivals
    trigger NPC reaction checks
    entering_player_id =
    UUID(payload["entity_id"])
    room_id = UUID(payload["room_id"])
    zone_id
    = UUID(payload["zone_id"])
    # Fetch all NPCs currently in the room
    occupants = await gt_client.get_room_occupants(room_id, entity_type="NPC")
    if not occupants:
        return
    logger.info(
        "Evaluating NPC
reactions to player arrival",
        room_id=room_id,
        player_id=entering_player_id,
        npc_count=len(occupants),
    )
    for npc_summary in occupants:
        npc_id = UUID(npc_summary["entity_id"])

```

```

npc = await gt_client.get_entity(npc_id) # Evaluate the NPC's
behavior tree with the new stimulus      decision = await
bt_engine.evaluate(npc=npc, stimulus={
"type": "PLAYER_ENTERED", "player_id":
str(entering_player_id), "room_id": str(room_id),
}, ) if decision is None: # BT did not fire
(e.g., FRIENDLY NPC, not triggered by player entry) continue
decision_payload = AgentDecisionPayload(npc_id=npc_id,
behavior_tree_node=decision.node_name,
decision_type=decision.decision_type,
decision_data=decision.data, confidence=decision.confidence,
) decision_evt =
(EnvelopeBuilder().for_type("agent.decision_made",
version="1.0.0").from_producer("agent_core",
producer_id=MY_POD_ID).caused_by(envelope) # causal:
world.room_entered → agent.decision_made .for_actor(npc_id)
.in_zone(zone_id).with_payload(decision_payload) .wi
th_ttl(60).build()) await
bus.publish(decision_evt) logger.info("NPC reaction
decision emitted", npc_id=npc_id,
decision_type=decision.decision_type,
envelope_id=decision_evt.envelope_id, )

```

Python 3.12 — agent_core/handlers/combat_started.py

```

""" agent_core – world.combat_started handler
===== Activates combat behavior trees
for all NPC participants in a new combat encounter and emits agent.goal_set
for each combat goal. """ from __future__ import annotations import logging
from uuid import UUID from nexusmud.events.envelope import EnvelopeBuilder,
EventEnvelope from nexusmud.groundtruth.client import GroundTruthClient from
nexusmud.agent_core.behavior import BehaviorTreeEngine logger =
logging.getLogger(__name__) MY_POD_ID: UUID = ... gt_client:
GroundTruthClient = ... bus: "EventBusProtocol" = ... bt_engine:
BehaviorTreeEngine = ... async def handle_world_combat_started(envelope:
EventEnvelope) -> None: """ Handler for world.combat_started events
in agent_core. For each NPC participant: 1. Switch the NPC's active
behavior tree to the combat BT. 2. Set a combat goal (e.g., ATTACK
primary target). 3. Emit agent.goal_set with the combat objective.
""" payload = envelope.payload participants: list[dict] =
payload.get("participants", []) zone_id = UUID(payload["zone_id"])
npc_participants = [p for p in participants if p.get("entity_type") == "NPC"]
if not npc_participants: return for npc_info in
npc_participants: npc_id = UUID(npc_info["entity_id"])
primary_target_id = UUID(payload.get("initiator_id",
npc_info.get("target_id", ""))) # Activate combat behavior tree
await bt_engine.activate_combat_tree(npc_id, target_id=primary_target_id)
# Emit agent.goal_set: NPC's combat objective goal_payload = {
"npc_id": str(npc_id), "goal_type": "COMBAT",
"goal_data": { "primary_target_id": str(primary_target_id),
"retreat_threshold_hp_pct": 0.15, # flee at 15% HP },
"priority": 100, # combat overrides all other goals }
goal_evt =
(EnvelopeBuilder().for_type("agent.goal_set",
version="1.0.0").from_producer("agent_core",
producer_id=MY_POD_ID).caused_by(envelope) # causal:
world.combat_started → agent.goal_set .for_actor(npc_id)
.in_zone(zone_id).with_payload(goal_payload) .with_t
tl(300).build()) await bus.publish(goal_evt)

```



```
logger.info("Combat goal set for NPC", npc_id=npc_id,
target=primary_target_id)
```

[Python 3.12 — agent_core/handlers/player_command_dialog.py](#)

```
""" agent_core – player.command_received handler (dialog routing)
===== Handles
commands directed at NPCs (e.g., "talk to innkeeper"). Initiates dialog
sessions and emits agent.dialog_started. """ from __future__ import
annotations import logging from uuid import UUID from
nexusmud.events.envelope import EnvelopeBuilder, EventEnvelope from
nexusmud.groundtruth.client import GroundTruthClient from
nexusmud.agent_core.dialog import DialogManager logger =
logging.getLogger(__name__) MY_POD_ID: UUID = ... gt_client:
GroundTruthClient = ... bus: "EventBusProtocol" = ... dialog_manager:
DialogManager = ... async def handle_player_command_for_dialog(envelope:
EventEnvelope) -> None: """ Handler for player.command_received in
agent_core. agent_core only processes commands that target NPCs for
dialog. Commands like MOVE, TAKE, ATTACK are handled by world_engine.
Both services subscribe to player.command_received and route internally.
""" payload = envelope.payload command_type: str =
payload.get("command_type", "") # Only handle TALK commands if
command_type != "TALK": return player_id =
UUID(payload["player_id"]) session_id = UUID(payload["session_id"])
target_name: str = payload.get("target_name", "").lower() room_id =
UUID(payload["room_id"]) # Resolve NPC by name within the room
occupants = await gt_client.get_room_occupants(room_id, entity_type="NPC")
target_npc = next( (o for o in occupants if o.get("display_name",
 "").lower() == target_name), None, ) if target_npc is None:
logger.info( "TALK target NPC not found in room",
target_name=target_name, room_id=room_id,
player_id=player_id, ) return # gateway_api handles "You
don't see that here." feedback npc_id = UUID(target_npc["entity_id"])
# Start the dialog session dialog_session_id = await
dialog_manager.start( npc_id=npc_id, player_id=player_id,
) # Emit agent.dialog_started – gateway_api will enter dialog mode
dialog_evt =
( EnvelopeBuilder().for_type("agent.dialog_started",
version="1.0.0").from_producer("agent_core", producer_id=MY_POD_ID)
.caused_by(envelope) # causal: player.command_received ->
agent.dialog_started .for_actor(npc_id) .for_session(session_id)
.with_payload({ "npc_id": str(npc_id),
"player_id": str(player_id), "session_id": str(session_id),
"dialog_session_id": str(dialog_session_id), "opening_line":
await dialog_manager.get_greeting(npc_id, ) }) .with_ttl(120)
.build() ) await bus.publish(dialog_evt) logger.info("Dialog
started", npc_id=npc_id, player_id=player_id)
```

2.10 Example Handlers: gateway_api

[Python 3.12 — gateway_api/handlers/room_entered_fanout.py](#)

```
""" gateway_api – world.room_entered handler
===== Fan-out: notify all player sessions
in the destination room of the new arrival, and send a room description to
```

```

the arriving player. """ from __future__ import annotations import logging
from uuid import UUID from nexusmud.events.envelope import EventEnvelope
from nexusmud.gateway_api.session_store import SessionStore from
nexusmud.gateway_api.renderer import RoomRenderer from
nexusmud.groundtruth.client import GroundTruthClient logger =
logging.getLogger(__name__) session_store: SessionStore = ... # maps
entity_id → WebSocket session gt_client: GroundTruthClient = ... renderer:
RoomRenderer = ... async def handle_world_room_entered(envelope:
EventEnvelope) -> None: """ Handler for world.room_entered in
gateway_api. Two responsibilities: 1. Notify all existing players in
the destination room about the arrival. 2. Send a full room description
(look) to the arriving player if entity_type=PLAYER. """ payload =
envelope.payload entity_id = UUID(payload["entity_id"]) entity_type:
str = payload.get("entity_type", "") room_id = UUID(payload["room_id"])
direction: str | None = payload.get("direction") # Resolve display name
of the arriving entity entity = await gt_client.get_entity(entity_id)
display_name: str = entity.get("display_name", "Someone") # --- Fan-out
arrival message to existing room occupants --- occupants = await
gt_client.get_room_occupants(room_id, entity_type="PLAYER") arrival_msg =
_format_arrival_message(display_name, entity_type, direction) for
occupant in occupants: occupant_id = UUID(occupant["entity_id"])
if occupant_id == entity_id: continue # Don't send arrival
message to the arriving player themselves session =
session_store.get_session(occupant_id) if session and
session.is_connected: await session.send_text(arrival_msg)
logger.debug("Sent arrival notice", to=occupant_id, arriving=entity_id)
# --- Send room description to the arriving player (PLAYER only) --- if
entity_type != "PLAYER": return arriving_session =
session_store.get_session(entity_id) if not arriving_session or not
arriving_session.is_connected: logger.warning( "Arriving
player has no active session",
player_id=entity_id,
room_id=room_id,
) return # Render full room
description: name, description, exits, occupants room_data = await
gt_client.get_entity(room_id) all_occupants = await
gt_client.get_room_occupants(room_id) room_text = await
renderer.render_room(room_data, all_occupants, viewer_id=entity_id) await
arriving_session.send_text(room_text) def _format_arrival_message(name:
str, entity_type: str, direction: str | None) -> str: if direction:
dir_labels = { "N": "the north", "S": "the south", "E": "the
east", "W": "the west", "NE": "the northeast", "NW": "the
northwest", "SE": "the southeast", "SW": "the
southwest", "UP": "above", "DOWN": "below", "IN": "inside",
"OUT": "outside", } from_label = dir_labels.get(direction,
"somewhere") return f"{name} arrives from {from_label}." return
f"{name} appears."

```

Python 3.12 — gateway_api/handlers/combat_started.py

```

""" gateway_api – world.combat_started handler
===== Transitions player sessions to
IN_COMBAT mode and begins delivering real-time combat updates. """ from
__future__ import annotations import logging from uuid import UUID from
nexusmud.events.envelope import EventEnvelope from
nexusmud.gateway_api.session_store import SessionStore, SessionState logger
= logging.getLogger(__name__) session_store: SessionStore = ... async def
handle_world_combat_started(envelope: EventEnvelope) -> None: """
Handler for world.combat_started in gateway_api. 1. Identify which
participants are players with active sessions. 2. Transition those
sessions to IN_COMBAT state. 3. Send combat start notification to each
involved player. """ payload = envelope.payload participants:

```

```

list[dict] = payload.get("participants", [])      combat_id: str =
payload.get("combat_id", str(envelope.envelope_id))    for participant in
participants:      if participant.get("entity_type") != "PLAYER":
continue      player_id = UUID(participant["entity_id"])      session =
session_store.get_session(player_id)      if not session or not
session.is_connected:      continue      # Transition session
state to IN_COMBAT      session.state = SessionState.IN_COMBAT
session.combat_id = combat_id      # Build and send combat start message
opponent_names = [      p.get("display_name", "Unknown")
for p in participants      if p.get("entity_id") != str(player_id)
]      opponents_str = " and ".join(opponent_names) if opponent_names else
"an enemy"      msg = f"* Combat begins! You are fighting {opponents_str}.
*\n"      msg += "Type ATTACK, FLEE, USE <item>, or CAST <spell>.\n"
await session.send_text(msg)      logger.info("Session transitioned to
IN_COMBAT", player_id=player_id, combat_id=combat_id)

```

[Python 3.12 — gateway_api/handlers/stat_changed.py](#)

```

""" gateway_api – player.stat_changed handler
===== Pushes stat delta updates to the
affected player's client. Handles HP=0 death sequence. """ from __future__
import annotations import logging from uuid import UUID from
nexusmud.events.envelope import EnvelopeBuilder, EventEnvelope from
nexusmud.gateway_api.session_store import SessionStore, SessionState logger
= logging.getLogger(__name__) MY_POD_ID: UUID = ... bus: "EventBusProtocol" =
... session_store: SessionStore = ... async def
handle_player_stat_changed(envelope: EventEnvelope) -> None:      """
Handler for player.stat_changed in gateway_api.      Pushes the updated stat
values to the player's client.      If hp reaches 0, triggers the death
sequence:      - Send death message to the dying player.      - Transition
session to DEAD state.      - The actual player.disconnected event is emitted
after the      death animation / respawn timer completes, not immediately
here.      """      payload = envelope.payload      player_id =
UUID(payload["player_id"])      stat_name: str = payload["stat_name"] # e.g.
"hp"      new_value: int = payload["new_value"]      max_value: int | None =
payload.get("max_value")      delta: int = payload.get("delta", 0)
session = session_store.get_session(player_id)      if not session or not
session.is_connected:      return # Player not connected; stat stored in
GroundTruth regardless      # Format stat update message      if max_value is
not None:      stat_line = f"[{stat_name.upper()}]:
{new_value}/{max_value}]"      else:      stat_line =
f"[{stat_name.upper()}]: {new_value}]"      if delta != 0:      sign = "+"
if delta > 0 else ""      stat_line += f" ({sign}{delta})"      await
session.send_text(stat_line)      # Death sequence      if stat_name == "hp"
and new_value <= 0:      logger.info("Player HP reached 0 – initiating
death sequence", player_id=player_id)      session.state =
SessionState.DEAD      await session.send_text(      "\n*** You
have been slain! ***\n"      "Your spirit rises from your body. You
will respawn shortly.\n"      )      # Note: player.disconnected is NOT
emitted here.      # The respawn manager (world_engine) owns the death →
respawn flow.      # gateway_api holds the session in DEAD state until
respawn is confirmed.

```

[Python 3.12 — gateway_api/handlers/level_up.py](#)

```

""" gateway_api – player.level_up handler
===== Delivers level-up notification to the
player, broadcasts to zone, and triggers reward delivery via GroundTruth. """
from __future__ import annotations import logging from uuid import UUID

```

```

from nexusmod.events.envelope import EventEnvelope from
nexusmod.gateway_api.session_store import SessionStore from
nexusmod.groundtruth.client import GroundTruthClient logger =
logging.getLogger(__name__) session_store: SessionStore = ... gt_client:
GroundTruthClient = ... async def handle_player_level_up(envelope:
EventEnvelope) -> None: """ Handler for player.level_up in
gateway_api. 1. Send a congratulatory message to the leveling player's
session. 2. Broadcast a zone-wide announcement to all connected players
in the zone. 3. Trigger reward delivery via a GroundTruth item transfer
call. """ payload = envelope.payload player_id =
UUID(payload["player_id"]) new_level: int = payload["new_level"]
display_name: str = payload.get("display_name", "A player") zone_id: str
| None = payload.get("zone_id") reward_item_ids: list[str] =
payload.get("reward_item_ids", []) # 1. Send personal level-up message
to the leveling player player_session =
session_store.get_session(player_id) if player_session and
player_session.is_connected: await player_session.send_text(
f"\n*** LEVEL UP! *** You are now level {new_level}!\n" f"New
abilities and stat increases have been applied.\n" ) # 2. Zone-
wide broadcast if zone_id: all_zone_sessions =
session_store.get_sessions_in_zone(zone_id) broadcast_msg = (
f"[Zone] {display_name} has reached level {new_level}! "
f"Congratulations!" ) for session in all_zone_sessions:
if session.player_id != player_id and session.is_connected:
await session.send_text(broadcast_msg) # 3. Trigger reward delivery via
GroundTruth for item_id_str in reward_item_ids: await
gt_client.transfer_item( item_id=UUID(item_id_str),
from_entity_id=None, # None = world / reward pool
to_entity_id=player_id, idempotency_key=f"level_up_reward:
{player_id}:{new_level}:{item_id_str}", ) logger.info("Level-
up reward delivered", player_id=player_id, item_id=item_id_str)
logger.info("Level-up handling complete", player_id=player_id,
new_level=new_level)

```

2.11 Versioning & Schema Evolution

Semver Rules for event_version

Version Segment	Increment When	Example
MAJOR	Breaking changes to the payload schema: removing a required field, changing a field's type, renaming a field, or changing the semantics of an existing field in an incompatible way.	1.2.0 → 2.0.0
MINOR	Backward-compatible additions: new optional fields added to the payload. Existing consumers that	1.0.0 → 1.1.0

Version Segment	Increment When	Example
	ignore unknown fields continue to work without modification.	
PATCH	Documentation corrections, default value fixes, or constraint clarifications that do not alter the wire format or consumer behavior.	1.1.0 → 1.1.1

Consumer Version Negotiation

Each consumer service declares a supported version range for each `event_type` it subscribes to in its service manifest (e.g., `world.room_entered: ">=1.0.0 <2.0.0"`). The event bus broker routes events to consumers based on compatibility. Events produced at a version outside a consumer's declared range are held in a staging queue until the consumer is upgraded, rather than delivered and potentially misprocessed.

Migration Path for Breaking Changes

17. **Dual-publishing period:** For a configurable transition window (default 30 days), producers emit both the old and new major versions simultaneously. The new version carries a different `event_version` MAJOR; consumers opt in to the new version by updating their manifest.
18. **Version adapters:** Consumers that cannot immediately upgrade may deploy a version adapter: a thin translation layer that converts new-version payloads to the old schema. Adapters are temporary and must be removed before the deprecation sunset date.
19. **Sunset:** After the transition window, dual-publishing stops. The old version is no longer produced. Any consumer still on the old version receives no new events until it upgrades.

Payload Migration Example: WorldRoomEnteredPayload

The following example shows the addition of an optional `weather_state` field — a MINOR (non-breaking) change.

[JSON — WorldRoomEnteredPayload v1.0.0 \(before migration\)](#)

```
{  "entity_id": "550e8400-e29b-41d4-a716-446655440000",  "room_id":
"550e8400-e29b-41d4-a716-446655440001",  "previous_room_id": "550e8400-e29b-
41d4-a716-446655440002",  "direction": "N",  "zone_id":  "550e8400-e29b-
41d4-a716-446655440003",  "entity_type": "PLAYER" }
```

JSON — WorldRoomEnteredPayload v1.1.0 (after migration, event_version bumped to 1.1.0)

```
{  "entity_id": "550e8400-e29b-41d4-a716-446655440000",  "room_id":
"550e8400-e29b-41d4-a716-446655440001",  "previous_room_id": "550e8400-e29b-
41d4-a716-446655440002",  "direction": "N",  "zone_id":  "550e8400-e29b-
41d4-a716-446655440003",  "entity_type": "PLAYER",  "weather_state":
"CLEAR"  // NEW optional field – v1.0.0 consumers safely ignore this }
```

Consumers on v1.0.0 that do not know about weather_state continue to function correctly because the field is optional and additive. No adapter is required. Once all consumers are updated to v1.1.0, the field becomes universally available.

Field Deprecation Path

20. Mark the field as deprecated in the schema registry with a deprecation notice and sunset MAJOR version.
21. Change the field from required to optional in the current MINOR release so producers may omit it.
22. Remove the field entirely in the next MAJOR version release.
23. Never remove a required field in a MINOR or PATCH release.

2.12 Error & Dead-Letter Handling

Retry Policy

Parameter	Value	Notes
Maximum retries	5	Per consumer group per envelope_id
Initial delay	500 ms	Before first retry
Backoff multiplier	2×	Exponential: 0.5s, 1s, 2s, 4s, 8s
Jitter	±20%	Applied to each delay to prevent thundering herd
Maximum delay	30 s	Capped regardless of

Parameter	Value	Notes
		backoff calculation
After max retries exceeded	Route to DLQ	Original envelope enriched with dlq_metadata

Dead-Letter Queue Structure

Every event bus topic has a corresponding DLQ topic (`{topic}.dlq`). When an envelope is routed to the DLQ, it is enriched with a `dlq_metadata` extension object:

JSON — DLQ Event Schema (enriched envelope)

```
{  // ... all original EventEnvelope fields preserved unchanged ...
"dlq_metadata": {    "failure_reason": "ValueError: entity_id 'bad-uuid' is
not a valid UUID",    "failure_type": "DESERIALIZATION_ERROR",
"retry_count": 5,    "first_attempted_at": "2026-05-31T13:00:00Z",
"last_attempted_at": "2026-05-31T13:04:07Z",    "consumer_id":
"world_engine:pod-abc123",    "consumer_group":
"world_engine.player_command_handlers",    "dlq_ingested_at": "2026-05-
31T13:04:08Z",    "is_poison_pill": false,    "poison_pill_detected_at":
null  } }
```

DLQ Metadata Field Definitions

Field	Description
failure_reason	The exception message or structured error string from the last processing attempt.
failure_type	Enum: DESERIALIZATION_ERROR VALIDATION_ERROR HANDLER_EXCEPTION TTL_EXPIRED POISON_PILL
retry_count	Total number of processing attempts made before routing to DLQ.
first_attempted_at	ISO 8601 UTC timestamp of the first processing attempt.
last_attempted_at	ISO 8601 UTC timestamp of the most recent failed attempt.
consumer_id	The specific pod and consumer instance that last attempted processing.
consumer_group	The consumer group identifier. Used to identify which service type failed.
is_poison_pill	True if the same envelope_id has failed on 3 or more distinct consumer instances.
poison_pill_detected_at	ISO 8601 UTC timestamp when poison pill classification was triggered. Null otherwise.

Poison Pill Detection

A *poison pill* is an event that consistently fails across multiple independent consumer instances, indicating a data-level defect rather than a transient infrastructure fault. Poison pill detection operates as follows:

24. Each failed processing attempt is recorded in the DLQ metadata store, keyed by `envelope_id`.
25. When the same `envelope_id` has failed on 3 or more distinct `consumer_id` values, the event is classified as a poison pill.
26. On classification: `is_poison_pill` is set to true in `dlq_metadata`; an alert is sent to the on-call engineering channel via PagerDuty with the full DLQ envelope attached.
27. The event remains in the DLQ. It will not be automatically replayed until a human operator confirms the root cause has been addressed.
28. Reprocessing is initiated manually via the DLQ management tooling after deploying a fix.

Reprocessing Workflow

29. **Inspect:** Engineer reviews the DLQ event in the observability dashboard. `failure_reason` and `causal_chain` provide root cause context.
30. **Fix:** Deploy a code fix or data correction to the affected consumer service.
31. **Replay:** Use the DLQ management CLI (`nexusmud-dlq replay --envelope-id {id}`) to re-queue the event on the original topic. The event's `flags.is_replay` is set to true to suppress non-idempotent side effects.
32. **Monitor:** Confirm the replayed event processes successfully in the observability dashboard before closing the incident.

Recommended Monitoring Metrics

Metric Name	Type	Labels	Description
event_processing_latency_ms	Histogram	consumer_service, event_type	End-to-end processing latency from published_at to handler completion. Alert P99 > 500 ms.
events_dlq_total	Counter	consumer_service, event_type, failure_type	Total events routed to DLQ. Alert on rate > 0 for POISON_PILL failure_type.
duplicate_events_total	Counter	consumer_service, event_type	Total duplicate events detected and skipped by IdempotencyGuard. Used to tune broker retry settings.
events_published_total	Counter	producer_service, event_type	Total events successfully published to the bus. Baseline for publish success rate.
publish_lag_ms	Histogram	producer_service	Delta between occurred_at and published_at. Alert P99 > 100 ms indicates broker pressure.

Appendix A: Glossary

Term	Definition
Entity	The base object type in NexusMUD. Every game object (Room, Player, NPC, Item, Zone) is an Entity with a globally unique entity_id and a monotonic version field managed by GroundTruth.
Event Envelope	The universal wrapper schema that all events on the NexusMUD event bus must conform to. Contains routing, observability, causal, and idempotency metadata in addition to the typed payload.
GroundTruth	The authoritative persistence service for all

Term	Definition
	durable world state. All services read from and write to GroundTruth; no service reads another service's internal database directly.
Producer	Any service that creates and publishes an EventEnvelope to the event bus. Producers are responsible for schema validity, correct causal metadata propagation, and idempotency key assignment.
Consumer	Any service that subscribes to an event type on the bus and processes received envelopes. Consumers are responsible for idempotency enforcement and retry handling.
Causal Chain	The ordered list of <code>envelope_ids</code> from a root event through all intermediate ancestors to the immediate parent. Stored in <code>causal_chain</code> and limited to 32 entries. Enables trace tree reconstruction.
Correlation ID	A UUID shared by all events originating from a single user request or business transaction. Enables grouping of all events in a request trace, regardless of branching.
Causation ID	The <code>envelope_id</code> of the single, immediately preceding event that caused the current event. Establishes parent-child relationships in an event trace tree.
Idempotency Key	A business-level string key used for deduplication. When present, takes precedence over <code>envelope_id</code> for duplicate detection. Allows re-emission with a new <code>envelope_id</code> without triggering double-processing.
Dead-Letter Queue (DLQ)	A secondary event queue where unprocessable or repeatedly-failing events are routed after exhausting retry attempts. DLQ events carry enriched diagnostic metadata for human review and replay.
Shard Key	The partition key used to route events to a specific partition of the event bus, ensuring ordered delivery for events with the same key. Derived from <code>zone_id</code> or <code>actor_id</code> if not explicitly set.
Event Bus	The asynchronous message-passing infrastructure (implemented on Kafka or NATS JetStream) through which all NexusMUD services communicate asynchronously. Guarantees at-least-once delivery.
Behavior Tree (BT)	A hierarchical AI decision-making structure

Term	Definition
	used by <code>agent_core</code> to control NPC behavior. BT nodes evaluate conditions and execute actions; the resolved output is an <code>AgentDecisionPayload</code> .
Zone	A logical grouping of Rooms that form a geographic region of the game world. Zones are the primary unit of shard partitioning and access control in NexusMUD.
Tick	A discrete time step in the world engine's simulation loop. Each tick processes NPC behavior, applies ongoing effects, and evaluates time-based events. Tick architecture is detailed in the planned Chapter 3.
Session	A persistent WebSocket connection between a player's client and <code>gateway_api</code> . Sessions carry state (<code>NORMAL</code> , <code>IN_COMBAT</code> , <code>IN_DIALOG</code> , <code>DEAD</code>) and are the delivery mechanism for all real-time player output.
world_engine	The core simulation service responsible for world state mutations, movement validation, combat resolution, and item management. Produces most <code>world.*</code> events and consumes <code>player.*</code> and <code>agent.*</code> events.
agent_core	The NPC AI service that runs behavior trees for all non-player characters. Consumes <code>world.*</code> events to detect stimuli and produces <code>agent.*</code> events encoding NPC decisions and goals.
gateway_api	The player-facing WebSocket service that manages player sessions, parses raw commands into <code>player.command_received</code> events, and fans out world events as formatted text to connected player clients.

Appendix B: Future Chapters (Placeholder)

The following chapters are planned for inclusion in future versions of this Design Companion. Each chapter will follow the same structural conventions established in Chapters 1 and 2.

Chapter	Title	Description	Target Version
3	World Engine Tick Architecture	Specifies the discrete simulation tick loop: tick rate, processing order, entity update batching, time-of-day simulation, and the interface between the tick system and the event bus for producing time-driven world events.	0.3.0
4	Agent Behavior Tree Specification	Defines the NexusMUD behavior tree DSL, the standard node library (selector, sequence, condition, action), the stimulus model consumed by BT evaluators, and the integration contract between agent_core and GroundTruth for NPC state reads.	0.4.0
5	Gateway WebSocket Protocol	Documents the WebSocket frame format, session handshake and authentication flow, command input parsing grammar, output rendering pipeline (text color codes, structured JSON mode), and the client-facing API for third-party client developers.	0.5.0
6	Zone Partitioning & Shard Management	Covers the zone-to-shard assignment algorithm, dynamic rebalancing protocol triggered by system.shard_rebalanced events, cross-shard entity migration for player movement across zone boundaries, and the shard health	0.6.0

Chapter	Title	Description	Target Version
		monitoring model.	
7	Analytics & Observability Pipeline	Describes the analytics service's event ingestion pipeline, the OpenTelemetry instrumentation standards for all NexusMUD services, the recommended Grafana dashboard schema, and the data retention and archival policy for event log and audit log data.	0.7.0

Contributing:

To propose a new chapter or section, open a design document in the internal engineering wiki under

`/design/nexusmud/proposals/`

. Proposals must include a one-paragraph scope statement, a section outline, and identification of the primary author and two reviewers.

Document Revision History

Version	Date	Author	Summary of Changes	Reviewers
0.1.0	2026-03-15	Platform Architecture Team	Initial document creation. Chapter 1 (GroundTruth API Specification)	Backend Lead, Security Lead

Version	Date	Author	Summary of Changes	Reviewers
			added in full: Entity data models, REST endpoints, authentication, error handling, rate limiting, and versioning strategy. Initial TOC established.	
0.2.0	2026-05-31	Platform Architecture Team	Chapter 2 (Event Envelope Specification) added in full: JSON Schema (draft-2020-12), field definitions, typed Python interface (Python 3.12), idempotency rules, causal lineage model, event type registry, handler examples for world_engine / agent_core / gateway_api, versioning rules, and DLQ/error handling. TOC updated. Appendix A (Glossary) and Appendix B (Future Chapters) added. Document status updated to Living Document.	Backend Lead, AI/NPC Lead, Gateway Lead

This document is subject to change. Always refer to the latest version in the internal documentation portal.